

Cost Manager | Final Project Submission

--- TEAM INFO ---

Team Manager: Eliya Zakay

Team Members:

1. Polina Katsnelson | ID: 342854940 | Phone: 0587514036 | Email: katsnelsonp@gmail.com
2. Sam Sotil | ID: 207970807 | Phone: 0544500290 | Email: sotilsam@gmail.com
3. Eliya Zakay | ID: 323094847 | Phone: 0545874150 | Email: eliyazak3@gmail.com

Video Link: <https://youtu.be/3IVD6tUVs6Y>

Collaborative Tools Used:

We used GitHub for version control and task management,
and WhatsApp for team communication and coordination.

--- CODE FILES ---

The following pages contain all source code files for the project.

src/App.jsx

```
1 // App.jsx
2 // Root component of the Cost Manager application.
3 // Manages navigation state and renders the active page.
4
5 import { useState, useMemo } from 'react';
6 import { Box, Container } from '@mui/material';
7 import NavBar from '../components/NavBar';
8 import AddCostForm from '../components/AddCostForm';
9 import MonthlyReport from '../components/MonthlyReport';
10 import PieChartView from '../components/PieChartView';
11 import BarChartView from '../components/BarChartView';
12 import Settings from '../components/Settings';
13 import { openCostsDB } from '../db';
14
15 // Constants for the database used throughout the application
16 const DB_NAME = 'costsdb';
17 const DB_VERSION = 1;
18
19 function App() {
20   // Track which page is currently visible
21   const [currentPage, setCurrentPage] = useState('add');
22
23   // Open the database once and reuse the same instance across renders
24   const dbInstance = useMemo(() => openCostsDB(DB_NAME, DB_VERSION), []);
25
26   // Return the correct component based on the active navigation tab
27   const renderPage = () => {
28     switch (currentPage) {
29       case 'add':
30         return <AddCostForm dbInstance={dbInstance} />;
31       case 'report':
32         return <MonthlyReport dbName={DB_NAME} />;
33       case 'pie':
34         return <PieChartView dbName={DB_NAME} />;
35       case 'bar':
36         return <BarChartView dbName={DB_NAME} />;
```

```
37         case 'settings':
38             return <Settings />;
39         default:
40             return <AddCostForm dbInstance={dbInstance} />;
41     }
42 };
43
44 return (
45     <Box sx={{ flexGrow: 1, minHeight: '100vh' }}>
46         {/* Navigation bar is always visible at the top */}
47         <NavBar currentPage={currentPage} onNavigate={setCurrentPage} />
48         <Container maxWidth='md' sx={{ mt: 4, pb: 4 }}>
49             {renderPage()}
50         </Container>
51     </Box>
52 );
53 }
54
55 export default App;
```

src/App.css

```
1 .counter {
2   font-size: 16px;
3   padding: 5px 10px;
4   border-radius: 5px;
5   color: var(--accent);
6   background: var(--accent-bg);
7   border: 2px solid transparent;
8   transition: border-color 0.3s;
9   margin-bottom: 24px;
10
11  &:hover {
12    border-color: var(--accent-border);
13  }
14  &:focus-visible {
15    outline: 2px solid var(--accent);
16    outline-offset: 2px;
17  }
18 }
19
20 .hero {
21   position: relative;
22
23   .base,
24   .framework,
25   .vite {
26     inset-inline: 0;
27     margin: 0 auto;
28   }
29
30   .base {
31     width: 170px;
32     position: relative;
33     z-index: 0;
34   }
35
36   .framework,
```

```
37 .vite {
38     position: absolute;
39 }
40
41 .framework {
42     z-index: 1;
43     top: 34px;
44     height: 28px;
45     transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
46     scale(1.4);
47 }
48
49 .vite {
50     z-index: 0;
51     top: 107px;
52     height: 26px;
53     width: auto;
54     transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
55     scale(0.8);
56 }
57 }
58
59 #center {
60     display: flex;
61     flex-direction: column;
62     gap: 25px;
63     place-content: center;
64     place-items: center;
65     flex-grow: 1;
66
67     @media (max-width: 1024px) {
68         padding: 32px 20px 24px;
69         gap: 18px;
70     }
71 }
72
73 #next-steps {
74     display: flex;
75     border-top: 1px solid var(--border);
```

```

76   text-align: left;
77
78   & > div {
79       flex: 1 1 0;
80       padding: 32px;
81       @media (max-width: 1024px) {
82           padding: 24px 20px;
83       }
84   }
85
86   .icon {
87       margin-bottom: 16px;
88       width: 22px;
89       height: 22px;
90   }
91
92   @media (max-width: 1024px) {
93       flex-direction: column;
94       text-align: center;
95   }
96 }
97
98 #docs {
99     border-right: 1px solid var(--border);
100
101     @media (max-width: 1024px) {
102         border-right: none;
103         border-bottom: 1px solid var(--border);
104     }
105 }
106
107 #next-steps ul {
108     list-style: none;
109     padding: 0;
110     display: flex;
111     gap: 8px;
112     margin: 32px 0 0;
113
114     .logo {

```

```

115     height: 18px;
116 }
117
118 a {
119     color: var(--text-h);
120     font-size: 16px;
121     border-radius: 6px;
122     background: var(--social-bg);
123     display: flex;
124     padding: 6px 12px;
125     align-items: center;
126     gap: 8px;
127     text-decoration: none;
128     transition: box-shadow 0.3s;
129
130     &:hover {
131         box-shadow: var(--shadow);
132     }
133     .button-icon {
134         height: 18px;
135         width: 18px;
136     }
137 }
138
139 @media (max-width: 1024px) {
140     margin-top: 20px;
141     flex-wrap: wrap;
142     justify-content: center;
143
144     li {
145         flex: 1 1 calc(50% - 8px);
146     }
147
148     a {
149         width: 100%;
150         justify-content: center;
151         box-sizing: border-box;
152     }
153 }

```

```
154 }
155
156 #spacer {
157   height: 88px;
158   border-top: 1px solid var(--border);
159   @media (max-width: 1024px) {
160     height: 48px;
161   }
162 }
163
164 .ticks {
165   position: relative;
166   width: 100%;
167
168   &::before,
169   &::after {
170     content: '';
171     position: absolute;
172     top: -4.5px;
173     border: 5px solid transparent;
174   }
175
176   &::before {
177     left: 0;
178     border-left-color: var(--border);
179   }
180   &::after {
181     right: 0;
182     border-right-color: var(--border);
183   }
184 }
```


src/index.css

```
1 /* index.css - Global base styles. MUI handles component-level styling. */
2
3 @import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Oxanium:wght@400;600;700;800&display=swap');
4
5 *,
6 *::before,
7 *::after {
8     box-sizing: border-box;
9 }
10
11 body {
12     margin: 0;
13     font-family: 'Inter', 'Roboto', 'Segoe UI', sans-serif;
14 }
```

src/main.jsx

```
1 // main.jsx
2 // Entry point for the React application.
3 // Sets up the MUI theme provider and mounts the App component.
4
5 import React from 'react';
6 import ReactDOM from 'react-dom/client';
7 import { ThemeProvider, createTheme } from '@mui/material/styles';
8 import CssBaseline from '@mui/material/CssBaseline';
9 import App from './App';
10 import './index.css';
11
12 // Define the global MUI theme ? dark glassmorphism design
13 const theme = createTheme({
14   palette: {
15     mode: 'dark',
16     primary: { main: '#90caf9' },
17     secondary: { main: '#ce93d8' },
18     background: { default: 'transparent', paper: '#1a2540' },
19     text: { primary: 'ffffff', secondary: 'rgba(255,255,255,0.7)' }
20   },
21   typography: {
22     fontFamily: '"Inter", "Roboto", "Helvetica", "Arial", sans-serif',
23     h5: { fontWeight: 700 },
24     h6: { fontWeight: 600 }
25   },
26   components: {
27     MuiCssBaseline: {
28       styleOverrides: {
29         body: {
30           background: 'linear-gradient(135deg, #1a1a2e 0%, #16213e 50%, #0f3460 100%)',
31           backgroundAttachment: 'fixed',
32           backgroundRepeat: 'no-repeat',
33           backgroundSize: 'cover',
34           minHeight: '100vh'
35         }
36       }
37     }
38   }
39 });
```

```

37     },
38     MuiPaper: {
39       styleOverrides: {
40         root: { backgroundImage: 'none', backgroundColor: '#1a2540' }
41       }
42     },
43     MuiCard: {
44       styleOverrides: {
45         root: {
46           backgroundImage: 'none',
47           background: 'rgba(255, 255, 255, 0.08)',
48           backdropFilter: 'blur(20px)',
49           WebkitBackdropFilter: 'blur(20px)',
50           border: '1px solid rgba(255, 255, 255, 0.15)',
51           borderRadius: 16,
52           boxShadow: '0 8px 32px rgba(0, 0, 0, 0.37)'
53         }
54       }
55     },
56     MuiOutlinedInput: {
57       styleOverrides: {
58         root: {
59           background: 'rgba(255, 255, 255, 0.07)',
60           borderRadius: 8,
61           '& .MuiOutlinedInput-notchedOutline': {
62             borderColor: 'rgba(255, 255, 255, 0.25)'
63           },
64           '&:hover .MuiOutlinedInput-notchedOutline': {
65             borderColor: 'rgba(255, 255, 255, 0.5)'
66           },
67           '& .Mui-focused .MuiOutlinedInput-notchedOutline': {
68             borderColor: 'rgba(255, 255, 255, 0.8)'
69           }
70         }
71       }
72     },
73     MuiInputLabel: {
74       styleOverrides: {
75         root: {

```

```

76         color: 'rgba(255, 255, 255, 0.6)',
77         '&.Mui-focused': { color: 'rgba(255, 255, 255, 0.9)' }
78     }
79 }
80 },
81 MuiSelect: {
82     styleOverrides: {
83         icon: { color: 'rgba(255, 255, 255, 0.6)' }
84     }
85 },
86 MuiButton: {
87     styleOverrides: {
88         contained: {
89             background: 'rgba(255, 255, 255, 0.15)',
90             backdropFilter: 'blur(10px)',
91             WebkitBackdropFilter: 'blur(10px)',
92             border: '1px solid rgba(255, 255, 255, 0.3)',
93             color: '#ffffff',
94             fontWeight: 600,
95             letterSpacing: '0.05em',
96             boxShadow: '0 4px 15px rgba(0, 0, 0, 0.2)',
97             '&:hover': {
98                 background: 'rgba(255, 255, 255, 0.28)',
99                 boxShadow: '0 6px 20px rgba(255, 255, 255, 0.15)',
100                transform: 'translateY(-1px)'
101            },
102            '&.Mui-disabled': {
103                background: 'rgba(255, 255, 255, 0.05)',
104                color: 'rgba(255, 255, 255, 0.3)',
105                border: '1px solid rgba(255, 255, 255, 0.1)'
106            },
107            transition: 'all 0.25s ease'
108        }
109    }
110 },
111 MuiTableCell: {
112     styleOverrides: {
113         root: {
114             borderBottom: '1px solid rgba(255, 255, 255, 0.1)',

```

```

115         color: 'rgba(255, 255, 255, 0.87)'
116     },
117     head: {
118         color: 'rgba(255, 255, 255, 0.55)',
119         fontWeight: 600,
120         fontSize: '0.72rem',
121         textTransform: 'uppercase',
122         letterSpacing: '0.08em'
123     }
124 }
125 },
126 MuiTableRow: {
127     styleOverrides: {
128         root: {
129             '&:hover': { backgroundColor: 'rgba(255, 255, 255, 0.04)' }
130         }
131     }
132 },
133 MuiDivider: {
134     styleOverrides: {
135         root: { borderColor: 'rgba(255, 255, 255, 0.15)' }
136     }
137 },
138 MuiCircularProgress: {
139     styleOverrides: {
140         root: { color: 'rgba(255, 255, 255, 0.8)' }
141     }
142 },
143 MuiAlert: {
144     styleOverrides: {
145         root: {
146             backdropFilter: 'blur(8px)',
147             WebkitBackdropFilter: 'blur(8px)',
148             borderRadius: 8,
149             border: '1px solid rgba(255, 255, 255, 0.1)'
150         }
151     }
152 }
153 }

```

```
154 });  
155  
156 ReactDOM.createRoot(document.getElementById('root')).render(  
157   <React.StrictMode>  
158     <ThemeProvider theme={theme}>  
159       {/* CssBaseline normalizes browser default styles across all browsers */}  
160       <CssBaseline />  
161       <App />  
162     </ThemeProvider>  
163   </React.StrictMode>  
164 );
```

src/db.js

```
1 // src/db.js
2 // ES module version of the Cost Manager database library.
3 // Used by React components via import statements.
4
5 // Default exchange rates relative to USD (e.g. ILS:3.4 means 3.4 ILS = 1 USD)
6 const DEFAULT_RATES = { USD: 1, GBP: 0.6, EURO: 0.7, ILS: 3.4 };
7
8 // localStorage keys for costs, cached rates, and the settings URL
9 const RATES_KEY = 'cm_exchange_rates';
10 const RATES_URL_KEY = 'cm_rates_url';
11
12 // The default URL for fetching exchange rates (static JSON hosted by the team)
13 const DEFAULT_RATES_URL = 'https://eliyazakay.github.io/costmanager-rates/rates.json';
14
15 /*
16  * Fetches the latest exchange rates from the configured URL and caches them
17  * in localStorage so that getReport can use them synchronously.
18  * Falls back to default rates if the fetch fails for any reason.
19  */
20 async function fetchAndCacheRates() {
21     // Read the user-configured URL from settings, or use the default
22     const url = localStorage.getItem(RATES_URL_KEY) || DEFAULT_RATES_URL;
23
24     try {
25         const response = await fetch(url);
26         // Throw an error if the server returned a non-OK status
27         if (!response.ok) {
28             throw new Error(`Rates fetch failed with status ${response.status}`);
29         }
30         const rates = await response.json();
31         // Persist the fetched rates so other functions can use them
32         localStorage.setItem(RATES_KEY, JSON.stringify(rates));
33         return rates;
34     } catch (error) {
35         // On failure, return whatever is cached or the built-in defaults
36         console.error('Could not fetch exchange rates:', error);
```

```

37     const cached = localStorage.getItem(RATES_KEY);
38     return cached ? JSON.parse(cached) : DEFAULT_RATES;
39 }
40 }
41
42 // Returns the most recently cached exchange rates, or the built-in defaults
43 function getCachedRates() {
44     const stored = localStorage.getItem(RATES_KEY);
45     if (stored) {
46         return JSON.parse(stored);
47     }
48     return DEFAULT_RATES;
49 }
50
51 /*
52  * Converts a monetary amount from one currency to another.
53  * All conversions route through USD as the common base currency.
54  * Rate meaning: rates[currency] units of that currency equal 1 USD.
55  */
56 function convertCurrency(amount, fromCurrency, toCurrency, rates) {
57     // Convert the source amount to USD, then to the target currency
58     const amountInUSD = amount / rates[fromCurrency];
59     return amountInUSD * rates[toCurrency];
60 }
61
62 /*
63  * Opens or initializes a named cost database stored in localStorage.
64  * Returns a database object exposing the addCost method.
65  * Kicks off an async fetch of the latest exchange rates in the background.
66  */
67 function openCostsDB(databaseName, databaseVersion) {
68     // Initialize with an empty array if this database does not exist yet
69     if (!localStorage.getItem(databaseName)) {
70         localStorage.setItem(databaseName, JSON.stringify([]));
71     }
72
73     // Refresh exchange rates in the background without blocking the caller
74     fetchAndCacheRates();
75

```



```

76 // Return the database object bound to this specific database name
77 return {
78     /*
79      * Adds a new cost item and persists it in localStorage.
80      * The current date is automatically attached to the item.
81      */
82     addCost: function (cost) {
83         const costs = JSON.parse(localStorage.getItem(databaseName) || '[]');
84         const now = new Date();
85
86         // Build the complete cost object including today's date
87         const newCost = {
88             sum: cost.sum,
89             currency: cost.currency,
90             category: cost.category,
91             description: cost.description,
92             date: {
93                 day: now.getDate(),
94                 month: now.getMonth() + 1,
95                 year: now.getFullYear()
96             }
97         };
98
99         // Save the updated costs array back to localStorage
100         costs.push(newCost);
101         localStorage.setItem(databaseName, JSON.stringify(costs));
102         return newCost;
103     }
104 };
105 }
106
107 /*
108 * Generates a cost report for a specific month, year, and display currency.
109 * Fetches the latest exchange rates from the server before calculating totals.
110 * Defaults to the current month and year when those arguments are omitted.
111 */
112 async function getReport(dbName, currency, year, month) {
113     const now = new Date();
114     // Use current year and month as defaults when not explicitly provided

```

```

115     const targetYear = (year !== undefined) ? year : now.getFullYear();
116     const targetMonth = (month !== undefined) ? month : (now.getMonth() + 1);
117
118     // Fetch fresh rates from the server before building the report
119     const rates = await fetchAndCacheRates();
120
121     // Load the full cost list from localStorage
122     const allCosts = JSON.parse(localStorage.getItem(dbName) || '[]');
123
124     // Keep only costs that belong to the requested month and year
125     const filteredCosts = allCosts.filter(
126         cost => cost.date.year === targetYear && cost.date.month === targetMonth
127     );
128
129     // Sum all costs after converting each one to the requested currency
130     let totalSum = 0;
131     filteredCosts.forEach(cost => {
132         totalSum += convertCurrency(cost.sum, cost.currency, currency, rates);
133     });
134
135     // Round the total to two decimal places
136     totalSum = Math.round(totalSum * 100) / 100;
137
138     return {
139         year: targetYear,
140         month: targetMonth,
141         costs: filteredCosts,
142         total: { currency: currency, sum: totalSum }
143     };
144 }
145
146 /*
147  * Generates a yearly summary showing total costs per month.
148  * Used by the Bar Chart feature.
149  */
150 async function getYearlySummary(dbName, currency, year) {
151     const targetYear = year !== undefined ? year : new Date().getFullYear();
152
153     // Fetch fresh exchange rates before calculating

```

```

154     const rates = await fetchAndCacheRates();
155     const allCosts = JSON.parse(localStorage.getItem(dbName) || '[]');
156
157     // Build a 12-entry array, one total per month
158     const monthlyTotals = Array.from({ length: 12 }, (_, i) => ({
159         month: i + 1,
160         total: 0
161     }));
162
163     // Accumulate costs into the correct month bucket
164     allCosts.forEach(cost => {
165         if (cost.date.year === targetYear) {
166             const index = cost.date.month - 1;
167             monthlyTotals[index].total += convertCurrency(
168                 cost.sum, cost.currency, currency, rates
169             );
170         }
171     });
172
173     // Round each monthly total to two decimal places
174     monthlyTotals.forEach(entry => {
175         entry.total = Math.round(entry.total * 100) / 100;
176     });
177
178     return { year: targetYear, currency: currency, months: monthlyTotals };
179 }
180
181 /*
182  * Saves a custom exchange rate URL to localStorage.
183  * This URL is used by all subsequent calls to fetchAndCacheRates.
184  */
185 function saveRatesUrl(url) {
186     localStorage.setItem(RATES_URL_KEY, url);
187 }
188
189 // Returns the currently saved exchange rate URL, or the default
190 function getRatesUrl() {
191     return localStorage.getItem(RATES_URL_KEY) || DEFAULT_RATES_URL;
192 }

```

```
193
194 export {
195     openCostsDB,
196     getReport,
197     getYearlySummary,
198     fetchAndCacheRates,
199     saveRatesUrl,
200     getRatesUrl,
201     getCachedRates
202 };
```

db.js (root - vanilla version)

```
1 // db.js
2 // Vanilla JavaScript library for the Cost Manager application.
3 // Attaches the db object to the global (window) scope via <script> tag.
4
5 /*
6  * Module pattern using an IIFE to encapsulate all internal state
7  * while exposing only the public API through the returned object.
8  * This avoids polluting the global scope beyond the single db property.
9  */
10 window.db = (function () {
11
12     // Tracks the currently active database name set by openCostsDB
13     let activeDBName = null;
14
15     // Default exchange rates relative to USD (e.g. ILS:3.4 means 3.4 ILS = 1 USD)
16     const DEFAULT_RATES = { USD: 1, GBP: 0.6, EURO: 0.7, ILS: 3.4 };
17
18     // localStorage key used to cache the most recently fetched exchange rates
19     const RATES_KEY = 'cm_exchange_rates';
20
21     // Retrieves cached exchange rates from localStorage, falling back to defaults
22     function getExchangeRates() {
23         const stored = localStorage.getItem(RATES_KEY);
24         // Return parsed rates if previously cached, otherwise use built-in defaults
25         if (stored) {
26             return JSON.parse(stored);
27         }
28         return DEFAULT_RATES;
29     }
30
31     /*
32     * Converts a monetary amount from one currency to another.
33     * All conversions route through USD as the common base currency.
34     * Rate meaning: rate[currency] units of that currency equal 1 USD.
35     */
36     function convertCurrency(amount, fromCurrency, toCurrency) {
```

```

37     const rates = getExchangeRates();
38     // Convert source amount to USD first, then to the target currency
39     const amountInUSD = amount / rates[fromCurrency];
40     return amountInUSD * rates[toCurrency];
41 }
42
43 /*
44  * Opens or initializes a named cost database stored in localStorage.
45  * Returns a database object exposing the addCost method.
46  */
47 function openCostsDB(databaseName, databaseVersion) {
48     // Store the active DB name so getReport can access the same data
49     activeDBName = databaseName;
50
51     // Initialize with an empty array if this database has not been created yet
52     if (!localStorage.getItem(databaseName)) {
53         localStorage.setItem(databaseName, JSON.stringify([]));
54     }
55
56     // Return the database object with the addCost method attached
57     return {
58         /*
59          * Adds a new cost item to the active database.
60          * The current date is automatically attached to the item.
61          */
62         addCost: function (cost) {
63             const costs = JSON.parse(localStorage.getItem(activeDBName) || '[]');
64             const now = new Date();
65
66             // Build the complete cost object including today's date
67             const newCost = {
68                 sum: cost.sum,
69                 currency: cost.currency,
70                 category: cost.category,
71                 description: cost.description,
72                 date: {
73                     day: now.getDate(),
74                     month: now.getMonth() + 1,
75                     year: now.getFullYear()

```

```

76         }
77     };
78
79     // Persist the updated costs array back to localStorage
80     costs.push(newCost);
81     localStorage.setItem(activeDBName, JSON.stringify(costs));
82     return newCost;
83 }
84 };
85 }
86
87 /*
88  * Generates a cost report for a specific month, year, and display currency.
89  * Defaults to the current month and year when those arguments are omitted.
90  * Uses cached exchange rates from localStorage for all currency conversions.
91  */
92 function getReport(currency, year, month) {
93     if (!activeDBName) {
94         throw new Error('No database is open. Call openCostsDB first.');
```

```
115
116     // Round the total to two decimal places for clean output
117     totalSum = Math.round(totalSum * 100) / 100;
118
119     return {
120         year: targetYear,
121         month: targetMonth,
122         costs: filteredCosts,
123         total: { currency: currency, sum: totalSum }
124     };
125 }
126
127 // Expose only the public API of the db module
128 return {
129     openCostsDB: openCostsDB,
130     getReport: getReport
131 };
132
133 }());
```


index.html

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>costmanager</title>
8   </head>
9   <body>
10    <div id="root"></div>
11    <script type="module" src="/src/main.jsx"></script>
12  </body>
13 </html>
```

vite.config.js

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vite.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7 })
```

package.json

```
1 {
2   "name": "costmanager",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "build": "vite build",
9     "lint": "eslint .",
10    "preview": "vite preview"
11  },
12  "dependencies": {
13    "@emotion/react": "^11.14.0",
14    "@emotion/styled": "^11.14.1",
15    "@mui/icons-material": "^9.0.1",
16    "@mui/material": "^9.0.1",
17    "react": "^19.2.6",
18    "react-dom": "^19.2.6",
19    "recharts": "^3.8.1"
20  },
21  "devDependencies": {
22    "@eslint/js": "^10.0.1",
23    "@types/react": "^19.2.14",
24    "@types/react-dom": "^19.2.3",
25    "@vitejs/plugin-react": "^6.0.1",
26    "eslint": "^10.3.0",
27    "eslint-plugin-react-hooks": "^7.1.1",
28    "eslint-plugin-react-refresh": "^0.5.2",
29    "globals": "^17.6.0",
30    "vite": "^8.0.12"
31  }
32 }
```

rates.json

```
1 {"USD":1, "GBP":0.6, "EURO":0.7, "ILS":3.4}
```

src/components/NavBar.jsx

```
1 // NavBar.jsx
2 // Top navigation bar for the Cost Manager application.
3 // Renders the app title and tab-based navigation links.
4
5 import { AppBar, Toolbar, Tabs, Tab, Typography } from '@mui/material';
6
7 // Navigation tab definitions with display labels and route keys
8 const NAV_TABS = [
9   { label: 'Add Cost', value: 'add' },
10  { label: 'Monthly Report', value: 'report' },
11  { label: 'Pie Chart', value: 'pie' },
12  { label: 'Bar Chart', value: 'bar' },
13  { label: 'Settings', value: 'settings' }
14 ];
15
16 /*
17  * NavBar renders the application's top AppBar with a tab set for navigation.
18  * It receives the current page key and a callback to change the active page.
19  */
20 function NavBar({ currentPage, onNavigate }) {
21   // Handle tab change events and propagate the new page key upward
22   const handleChange = (event, newValue) => {
23     onNavigate(newValue);
24   };
25
26   return (
27     <AppBar
28       position='static'
29       elevation={0}
30       sx={{
31         background: 'rgba(255, 255, 255, 0.08)',
32         backdropFilter: 'blur(12px)',
33         WebkitBackdropFilter: 'blur(12px)',
34         borderBottom: '1px solid rgba(255, 255, 255, 0.15)',
35         boxShadow: 'none'
36       }}
37     />
```

```

37     >
38     <Toolbar sx={{ gap: 1 }}>
39         <Typography
40             sx={{
41                 mr: 3,
42                 fontFamily: '"Oxanium", "Inter", sans-serif',
43                 fontWeight: 700,
44                 fontSize: '1.75rem',
45                 letterSpacing: '0.06em',
46                 color: '#fff',
47                 userSelect: 'none',
48                 textTransform: 'uppercase',
49                 textShadow: `
50                     0 0 6px rgba(255, 255, 255, 1),
51                     0 0 14px rgba(255, 255, 255, 0.85),
52                     0 0 28px rgba(255, 255, 255, 0.65),
53                     0 0 55px rgba(255, 255, 255, 0.35)
54                 `
55             }}
56         >
57         Spendly
58     </Typography>
59
60     {/* Navigation tabs aligned to the right of the title */}
61     <Tabs
62         value={currentPage}
63         onChange={handleChange}
64         textColor='inherit'
65         TabIndicatorProps={{
66             style: { backgroundColor: '#90caf9', height: 3, borderRadius: '3px 3px 0 0' }
67         }}
68         variant='scrollable'
69         scrollButtons='auto'
70     >
71         {NAV_TABS.map(tab => (
72             <Tab
73                 key={tab.value}
74                 label={tab.label}
75                 value={tab.value}

```

```
76         sx={{
77             letterSpacing: '0.04em',
78             fontWeight: 500,
79             opacity: 0.72,
80             px: 2,
81             '&.Mui-selected': { opacity: 1, fontWeight: 600 }
82         }}
83     />
84     )}}
85     </Tabs>
86     </Toolbar>
87     </AppBar>
88 );
89 }
90
91 export default NavBar;
```

src/components/AddCostForm.jsx

```
1 // AddCostForm.jsx
2 // Form component that allows the user to add a new cost item.
3 // On submit the cost is persisted to localStorage via the db module.
4
5 import { useState } from 'react';
6 import {
7   Box, Button, Card, CardContent, FormControl,
8   InputLabel, MenuItem, Select, TextField, Typography, Alert
9 } from '@mui/material';
10
11 // Supported currencies as required by the project specification
12 const CURRENCIES = ['USD', 'ILS', 'GBP', 'EURO'];
13
14 // Available cost categories for the user to choose from
15 const CATEGORIES = [
16   'Food', 'Education', 'Health', 'Entertainment',
17   'Transportation', 'Shopping', 'Utilities', 'Other'
18 ];
19
20 /*
21  * AddCostForm renders a card with input fields for sum, currency,
22  * category, and description. On submit it calls dbInstance.addCost
23  * and shows a success or error message to the user.
24  */
25 function AddCostForm({ dbInstance }) {
26   // Form field state
27   const [sum, setSum] = useState('');
28   const [currency, setCurrency] = useState('USD');
29   const [category, setCategory] = useState('');
30   const [description, setDescription] = useState('');
31
32   // Feedback message shown after a submit attempt
33   const [message, setMessage] = useState(null);
34
35   // Resets all form fields to their initial empty state
36   const resetForm = () => {
```



```

37     setSum('');
38     setCurrency('USD');
39     setCategory('');
40     setDescription('');
41 };
42
43 // Validates inputs and adds the cost item to the database
44 const handleSubmit = (event) => {
45     event.preventDefault();
46     setMessage(null);
47
48     // Basic validation: sum must be a positive number
49     const parsedSum = Number(sum);
50     if (!sum || parsedSum <= 0 || isNaN(parsedSum)) {
51         setMessage({ type: 'error', text: 'Please enter a valid positive sum.' });
52         return;
53     }
54
55     // Category selection is required
56     if (!category) {
57         setMessage({ type: 'error', text: 'Please select a category.' });
58         return;
59     }
60
61     // Description must not be empty
62     if (!description.trim()) {
63         setMessage({ type: 'error', text: 'Please enter a description.' });
64         return;
65     }
66
67     // Persist the new cost item using the db module
68     const result = dbInstance.addCost({
69         sum: parsedSum,
70         currency: currency,
71         category: category,
72         description: description.trim()
73     });
74
75     // Show success feedback and reset the form

```

```

76     if (result) {
77         setMessage({ type: 'success', text: 'Cost item added successfully!' });
78         resetForm();
79     }
80 };
81
82 return (
83     <Card elevation={0}>
84         <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
85             <Typography variant='h5' gutterBottom>
86                 Add New Cost
87             </Typography>
88
89             {/* Show feedback alert when a message exists */}
90             {message && (
91                 <Alert severity={message.type} sx={{ mb: 2 }}>
92                     {message.text}
93                 </Alert>
94             )}
95
96             <Box component='form' onSubmit={handleSubmit} noValidate>
97                 {/* Sum and Currency on the same row */}
98                 <Box sx={{ display: 'flex', gap: 2, mb: 2 }}>
99                     <TextField
100                         label='Sum'
101                         type='number'
102                         value={sum}
103                         onChange={e => setSum(e.target.value)}
104                         required
105                         fullWidth
106                         inputProps={{ min: 0, step: '0.01' }}
107                     />
108
109                     {/* Currency selector */}
110                     <FormControl fullWidth required>
111                         <InputLabel>Currency</InputLabel>
112                         <Select
113                             value={currency}
114                             label='Currency'

```

```

115         onChange={e => setCurrency(e.target.value)}
116     >
117         {CURRENCIES.map(c => (
118             <MenuItem key={c} value={c}>{c}</MenuItem>
119         ))}
120     </Select>
121 </FormControl>
122 </Box>
123
124     {/* Category selector */}
125     <FormControl fullWidth required sx={{ mb: 2 }}>
126         <InputLabel>Category</InputLabel>
127         <Select
128             value={category}
129             label='Category'
130             onChange={e => setCategory(e.target.value)}
131         >
132             {CATEGORIES.map(cat => (
133                 <MenuItem key={cat} value={cat}>{cat}</MenuItem>
134             ))}
135         </Select>
136     </FormControl>
137
138     {/* Description text field */}
139     <TextField
140         label='Description'
141         value={description}
142         onChange={e => setDescription(e.target.value)}
143         required
144         fullWidth
145         multiline
146         rows={2}
147         sx={{ mb: 3 }}
148     />
149
150     <Button
151         type='submit'
152         variant='contained'
153         size='large'

```

```
154         fullWidth
155     >
156         Add Cost
157     </Button>
158 </Box>
159 </CardContent>
160 </Card>
161 );
162 }
163
164 export default AddCostForm;
```

src/components/MonthlyReport.jsx

```
1 // MonthlyReport.jsx
2 // Displays a detailed cost report for a user-selected month, year, and currency.
3 // Fetches fresh exchange rates before calculating the total.
4
5 import { useState } from 'react';
6 import {
7   Box, Button, Card, CardContent, CircularProgress, FormControl,
8   InputLabel, MenuItem, Select, Table, TableBody, TableCell,
9   TableHead, TableRow, Typography, Alert, Divider
10 } from '@mui/material';
11 import { getReport } from '../db';
12
13 // Supported display currencies
14 const CURRENCIES = ['USD', 'ILS', 'GBP', 'EURO'];
15
16 // Month names used to build the month selector
17 const MONTH_NAMES = [
18   'January', 'February', 'March', 'April', 'May', 'June',
19   'July', 'August', 'September', 'October', 'November', 'December'
20 ];
21
22 // Build a list of recent years for the year selector (current year ± 4)
23 const buildYearOptions = () => {
24   const currentYear = new Date().getFullYear();
25   return Array.from({ length: 5 }, (_, i) => currentYear - i);
26 };
27
28 /*
29  * MonthlyReport allows the user to select a month, year, and display currency,
30  * then fetches and displays all costs for that period along with the total.
31  */
32 function MonthlyReport({ dbName }) {
33   const now = new Date();
34
35   // Selector state ? default to the current month, year, and USD
36   const [month, setMonth] = useState(now.getMonth() + 1);
```

```

37     const [year, setYear] = useState(now.getFullYear());
38     const [currency, setCurrency] = useState('USD');
39
40     // Report data and UI state
41     const [reportData, setReportData] = useState(null);
42     const [loading, setLoading] = useState(false);
43     const [error, setError] = useState(null);
44
45     const yearOptions = buildYearOptions();
46
47     // Fetches the report from the db module and updates state
48     const handleGetReport = async () => {
49         setLoading(true);
50         setError(null);
51         setReportData(null);
52
53         try {
54             const data = await getReport(dbName, currency, year, month);
55             setReportData(data);
56         } catch (err) {
57             setError('Failed to load report. Please try again.');
```

console.error(err);

```

59         } finally {
60             setLoading(false);
61         }
62     };
63
64     return (
65         <Card elevation={0}>
66             <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
67                 <Typography variant='h5' gutterBottom>
68                     Monthly Report
69                 </Typography>
70
71                 {/* Selectors row: month, year, and display currency */}
72                 <Box sx={{ display: 'flex', gap: 2, mb: 3, flexWrap: 'wrap' }}>
73                     <FormControl sx={{ minWidth: 150 }}>
74                         <InputLabel>Month</InputLabel>
75                         <Select
```

```

76         value={month}
77         label='Month'
78         onChange={e => setMonth(Number(e.target.value))}
79     >
80         {MONTH_NAMES.map((name, index) => (
81             <MenuItem key={name} value={index + 1}>{name}</MenuItem>
82         ))}
83     </Select>
84 </FormControl>
85
86     {/* Year selector */}
87     <FormControl sx={{ minWidth: 120 }}>
88         <InputLabel>Year</InputLabel>
89         <Select
90             value={year}
91             label='Year'
92             onChange={e => setYear(Number(e.target.value))}
93         >
94             {yearOptions.map(y => (
95                 <MenuItem key={y} value={y}>{y}</MenuItem>
96             ))}
97         </Select>
98     </FormControl>
99
100    {/* Display currency selector */}
101    <FormControl sx={{ minWidth: 120 }}>
102        <InputLabel>Currency</InputLabel>
103        <Select
104            value={currency}
105            label='Currency'
106            onChange={e => setCurrency(e.target.value)}
107        >
108            {CURRENCIES.map(c => (
109                <MenuItem key={c} value={c}>{c}</MenuItem>
110            ))}
111        </Select>
112    </FormControl>
113
114    <Button

```

```

115         variant='contained'
116         onClick={handleGetReport}
117         disabled={loading}
118         sx={{ height: 56, whiteSpace: 'nowrap' }}
119     >
120         Get Report
121     </Button>
122 </Box>
123
124     {/* Loading spinner shown while fetching */}
125     {loading && (
126         <Box sx={{ display: 'flex', justifyContent: 'center', mt: 3 }}>
127             <CircularProgress />
128         </Box>
129     )}
130
131     {/* Error alert shown on fetch failure */}
132     {error && <Alert severity='error'>{error}</Alert>}
133
134     {/* Report results table */}
135     {reportData && (
136         <Box>
137             <Divider sx={{ mb: 2 }} />
138
139             {reportData.costs.length === 0 ? (
140                 <Typography color='text.secondary'>
141                     No costs found for {MONTH_NAMES[month - 1]} {year}.
142                 </Typography>
143             ) : (
144                 <Table size='small'>
145                     <TableHead>
146                         <TableRow>
147                             <TableCell><strong>Day</strong></TableCell>
148                             <TableCell><strong>Category</strong></TableCell>
149                             <TableCell><strong>Description</strong></TableCell>
150                             <TableCell align='right'><strong>Sum</strong></TableCell>
151                             <TableCell><strong>Currency</strong></TableCell>
152                         </TableRow>
153                     </TableHead>

```



```

154         <TableBody>
155             {/* Render one row per cost item */}
156             {reportData.costs.map((cost, index) => (
157                 <TableRow key={index}>
158                     <TableCell>{cost.date.day}</TableCell>
159                     <TableCell>{cost.category}</TableCell>
160                     <TableCell>{cost.description}</TableCell>
161                     <TableCell align='right'>{cost.sum}</TableCell>
162                     <TableCell>{cost.currency}</TableCell>
163                 </TableRow>
164             )})
165         </TableBody>
166     </Table>
167 }}
168
169     {/* Total row displayed below the table */}
170     <Box sx={{ mt: 2, p: 2, background: 'rgba(144, 202, 249, 0.15)', border: '1px solid rgba(144, 202, 249, 0.35)', borderRadius: 2 }}>
171         <Typography variant='h6'>
172             Total: {reportData.total.sum} {reportData.total.currency}
173         </Typography>
174     </Box>
175 </Box>
176 }}
177 </CardContent>
178 </Card>
179 );
180 }
181
182 export default MonthlyReport;

```

src/components/BarChartView.jsx

```
1 // BarChartView.jsx
2 // Displays a bar chart of total monthly costs for a user-selected year and currency.
3 // Each bar represents the total spending for one of the twelve months.
4
5 import { useState } from 'react';
6 import {
7   Box, Button, Card, CardContent, CircularProgress,
8   FormControl, InputLabel, MenuItem, Select, Typography, Alert
9 } from '@mui/material';
10 import BarChartIcon from '@mui/icons-material/BarChart';
11 import {
12   BarChart, Bar, XAxis, YAxis, CartesianGrid,
13   Tooltip, ResponsiveContainer, Cell
14 } from 'recharts';
15 import { getYearlySummary } from '../db';
16
17 // Supported display currencies
18 const CURRENCIES = ['USD', 'ILS', 'GBP', 'EURO'];
19
20 // Short month labels for the X-axis of the bar chart
21 const MONTH_LABELS = [
22   'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
23   'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'
24 ];
25
26 const BAR_COLOR = '#64b5f6';
27
28 // Build a list of recent years for the year selector
29 const buildYearOptions = () => {
30   const currentYear = new Date().getFullYear();
31   return Array.from({ length: 5 }, (_, i) => currentYear - i);
32 };
33
34 /*
35  * BarChartView lets the user pick a year and currency, then renders a
36  * Recharts BarChart showing the total spending per month for that year.
```

```

37  */
38  function BarChartView({ dbName }) {
39      const now = new Date();
40
41      // Selector state ? default to the current year and USD
42      const [year, setYear] = useState(now.getFullYear());
43      const [currency, setCurrency] = useState('USD');
44
45      // Chart data and UI state
46      const [chartData, setChartData] = useState(null);
47      const [loading, setLoading] = useState(false);
48      const [error, setError] = useState(null);
49
50      const yearOptions = buildYearOptions();
51
52      // Fetches the yearly summary and prepares data for Recharts
53      const handleGetChart = async () => {
54          setLoading(true);
55          setError(null);
56          setChartData(null);
57
58          try {
59              const summary = await getYearlySummary(dbName, currency, year);
60
61              // Map the monthly totals to the format Recharts expects
62              const data = summary.months.map((entry, index) => ({
63                  month: MONTH_LABELS[index],
64                  total: entry.total
65              }));
66
67              setChartData(data);
68          } catch (err) {
69              setError('Failed to load chart data. Please try again.');
```

```

76     return (
77         <Card elevation={0}>
78             <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
79                 <Box sx={{ display: 'flex', alignItems: 'center', gap: 1, mb: 2 }}>
80                     <BarChartIcon color='primary' sx={{ fontSize: 28 }} />
81                     <Typography variant='h5'>
82                         Monthly Totals - Bar Chart
83                     </Typography>
84                 </Box>
85
86                 {/* Selectors row: year and currency */}
87                 <Box sx={{ display: 'flex', gap: 2, mb: 3, flexWrap: 'wrap' }}>
88                     <FormControl sx={{ minWidth: 120 }}>
89                         <InputLabel>Year</InputLabel>
90                         <Select
91                             value={year}
92                             label='Year'
93                             onChange={e => setYear(Number(e.target.value))}
94                         >
95                             {yearOptions.map(y => (
96                                 <MenuItem key={y} value={y}>{y}</MenuItem>
97                             ))}
98                         </Select>
99                     </FormControl>
100
101                     {/* Currency selector for converting all totals */}
102                     <FormControl sx={{ minWidth: 120 }}>
103                         <InputLabel>Currency</InputLabel>
104                         <Select
105                             value={currency}
106                             label='Currency'
107                             onChange={e => setCurrency(e.target.value)}
108                         >
109                             {CURRENCIES.map(c => (
110                                 <MenuItem key={c} value={c}>{c}</MenuItem>
111                             ))}
112                         </Select>
113                     </FormControl>
114

```

```

115         <Button
116             variant='contained'
117             onClick={handleGetChart}
118             disabled={loading}
119             sx={{ height: 56, whiteSpace: 'nowrap' }}
120         >
121             Show Chart
122         </Button>
123     </Box>
124
125     {/* Loading indicator */}
126     {loading && (
127         <Box sx={{ display: 'flex', justifyContent: 'center', mt: 3 }}>
128             <CircularProgress />
129         </Box>
130     )}
131
132     {/* Error feedback */}
133     {error && <Alert severity='error'>{error}</Alert>}
134
135     {/* Bar chart rendered once data is ready */}
136     {chartData && (
137         <ResponsiveContainer width='100%' height={400}>
138             <BarChart
139                 data={chartData}
140                 margin={{ top: 10, right: 30, left: 10, bottom: 5 }}
141             >
142                 <CartesianGrid strokeDasharray='3 3' stroke='rgba(255,255,255,0.08)' />
143                 <XAxis
144                     dataKey='month'
145                     tick={{ fill: 'rgba(255,255,255,0.7)', fontSize: 12 }}
146                     axisLine={{ stroke: 'rgba(255,255,255,0.2)' }}
147                     tickLine={{ stroke: 'rgba(255,255,255,0.2)' }}
148                 />
149                 <YAxis
150                     tick={{ fill: 'rgba(255,255,255,0.7)', fontSize: 12 }}
151                     axisLine={{ stroke: 'rgba(255,255,255,0.2)' }}
152                     tickLine={{ stroke: 'rgba(255,255,255,0.2)' }}
153                     label={{

```

```

154         value: currency,
155         angle: -90,
156         position: 'insideLeft',
157         offset: -5,
158         fill: 'rgba(255,255,255,0.6)'
159     }}
160     />
161     <Tooltip
162         formatter={value => [`${value} ${currency}`, 'Total']}
163         contentStyle={{
164             background: 'rgba(10, 20, 50, 0.92)',
165             border: '1px solid rgba(255,255,255,0.2)',
166             borderRadius: 8,
167             color: 'white'
168         }}
169         itemStyle={{ color: 'rgba(255,255,255,0.87)' }}
170         labelStyle={{ color: 'rgba(255,255,255,0.6)', marginBottom: 4 }}
171     />
172     <Bar dataKey='total' name='Total'>
173         {chartData.map((_, index) => (
174             <Cell key={`bar-${index}`} fill={BAR_COLOR} />
175         ))}
176     </Bar>
177 </BarChart>
178 </ResponsiveContainer>
179     )}
180 </CardContent>
181 </Card>
182 );
183 }
184
185 export default BarChartView;

```

src/components/PieChartView.jsx

```
1 // PieChartView.jsx
2 // Displays a pie chart of total costs grouped by category
3 // for a user-selected month, year, and display currency.
4
5 import { useState } from 'react';
6 import {
7   Box, Button, Card, CardContent, CircularProgress,
8   FormControl, InputLabel, MenuItem, Select, Typography, Alert
9 } from '@mui/material';
10 import { PieChart, Pie, Cell, Tooltip, Legend, ResponsiveContainer } from 'recharts';
11 import { getReport } from '../db';
12
13 // Supported display currencies
14 const CURRENCIES = ['USD', 'ILS', 'GBP', 'EURO'];
15
16 // Month names for the month selector
17 const MONTH_NAMES = [
18   'January', 'February', 'March', 'April', 'May', 'June',
19   'July', 'August', 'September', 'October', 'November', 'December'
20 ];
21
22 const SLICE_COLORS = [
23   '#64b5f6', '#ef5350', '#66bb6a', '#ffa726',
24   '#ab47bc', '#26c6da', '#ec407a', '#8d6e63'
25 ];
26
27 // Build a list of recent years for the year selector
28 const buildYearOptions = () => {
29   const currentYear = new Date().getFullYear();
30   return Array.from({ length: 5 }, (_, i) => currentYear - i);
31 };
32
33 /*
34  * Aggregates a flat list of cost items into totals grouped by category.
35  * Returns an array of { name, value } objects ready for Recharts.
36  */
```

```

37 function buildChartData(costs, currency, rates) {
38     const totals = {};
39
40     // Sum each cost into the correct category bucket
41     costs.forEach(cost => {
42         const amountInUSD = cost.sum / rates[cost.currency];
43         const converted = amountInUSD * rates[currency];
44
45         if (!totals[cost.category]) {
46             totals[cost.category] = 0;
47         }
48         totals[cost.category] += converted;
49     });
50
51     // Convert the totals map to the array format Recharts expects
52     return Object.entries(totals).map(([name, value]) => ({
53         name,
54         value: Math.round(value * 100) / 100
55     }));
56 }
57
58 /*
59  * PieChartView lets the user pick a month, year, and currency,
60  * then renders a Recharts PieChart grouped by cost category.
61  */
62 function PieChartView({ dbName }) {
63     const now = new Date();
64
65     // Selector state ? default to the current month and year
66     const [month, setMonth] = useState(now.getMonth() + 1);
67     const [year, setYear] = useState(now.getFullYear());
68     const [currency, setCurrency] = useState('USD');
69
70     // Chart data and UI state
71     const [chartData, setChartData] = useState(null);
72     const [loading, setLoading] = useState(false);
73     const [error, setError] = useState(null);
74
75     const yearOptions = buildYearOptions();

```



```

76
77 // Fetches the monthly report and transforms it into chart data
78 const handleGetChart = async () => {
79     setLoading(true);
80     setError(null);
81     setChartData(null);
82
83     try {
84         const reportData = await getReport(dbName, currency, year, month);
85
86         // Use cached rates (already fetched inside getReport) for conversion
87         const cached = localStorage.getItem('cm_exchange_rates');
88         const rates = cached
89             ? JSON.parse(cached)
90             : { USD: 1, GBP: 0.6, EURO: 0.7, ILS: 3.4 };
91
92         const data = buildChartData(reportData.costs, currency, rates);
93         setChartData(data);
94     } catch (err) {
95         setError('Failed to load chart data. Please try again.');
```

console.error(err);

```

97     } finally {
98         setLoading(false);
99     }
100 };
101
102 return (
103     <Card elevation={0}>
104         <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
105             <Typography variant='h5' gutterBottom>
106                 Costs by Category - Pie Chart
107             </Typography>
108
109             {/* Selectors row */}
110             <Box sx={{ display: 'flex', gap: 2, mb: 3, flexWrap: 'wrap' }}>
111                 <FormControl sx={{ minWidth: 150 }}>
112                     <InputLabel>Month</InputLabel>
113                     <Select
114                         value={month}

```

```

115         label='Month'
116         onChange={e => setMonth(Number(e.target.value))}
117     >
118         {MONTH_NAMES.map((name, index) => (
119             <MenuItem key={name} value={index + 1}>{name}</MenuItem>
120         ))}
121     </Select>
122 </FormControl>
123
124     {/* Year selector */}
125     <FormControl sx={{ minWidth: 120 }}>
126         <InputLabel>Year</InputLabel>
127         <Select
128             value={year}
129             label='Year'
130             onChange={e => setYear(Number(e.target.value))}
131         >
132             {yearOptions.map(y => (
133                 <MenuItem key={y} value={y}>{y}</MenuItem>
134             ))}
135         </Select>
136     </FormControl>
137
138     {/* Currency selector */}
139     <FormControl sx={{ minWidth: 120 }}>
140         <InputLabel>Currency</InputLabel>
141         <Select
142             value={currency}
143             label='Currency'
144             onChange={e => setCurrency(e.target.value)}
145         >
146             {CURRENCIES.map(c => (
147                 <MenuItem key={c} value={c}>{c}</MenuItem>
148             ))}
149         </Select>
150     </FormControl>
151
152     <Button
153         variant='contained'

```

```

154         onClick={handleGetChart}
155         disabled={loading}
156         sx={{ height: 56, whiteSpace: 'nowrap' }}
157     >
158         Show Chart
159     </Button>
160 </Box>
161
162     {/* Loading spinner */}
163     {loading && (
164         <Box sx={{ display: 'flex', justifyContent: 'center', mt: 3 }}>
165             <CircularProgress />
166         </Box>
167     )}
168
169     {/* Error message */}
170     {error && <Alert severity='error'>{error}</Alert>}
171
172     {/* Pie chart rendered when data is available */}
173     {chartData && chartData.length === 0 && (
174         <Typography color='text.secondary'>
175             No costs found for {MONTH_NAMES[month - 1]} {year}.
176         </Typography>
177     )}
178
179     {chartData && chartData.length > 0 && (
180         <ResponsiveContainer width='100%' height={400}>
181             <PieChart>
182                 <Pie
183                     data={chartData}
184                     dataKey='value'
185                     nameKey='name'
186                     cx='50%'
187                     cy='50%'
188                     outerRadius={140}
189                     labelLine={{ stroke: 'rgba(255,255,255,0.4)' }}
190                     label={({ name, percent, x, y }) => (
191                         <text
192                             x={x}

```

```

193             y={y}
194             fill='rgba(255,255,255,0.9)'
195             textAnchor='middle'
196             dominantBaseline='central'
197             fontSize={12}
198         >
199             `${name} ${percent * 100}.toFixed(1)}%`
200         </text>
201     )}
202 >
203     {chartData.map((entry, index) => (
204         <Cell
205             key={`cell-${index}`}
206             fill={SLICE_COLORS[index % SLICE_COLORS.length]}
207         />
208     ))}
209 </Pie>
210 <Tooltip
211     formatter={value => `${value} ${currency}`}
212     contentStyle={{
213         background: 'rgba(10, 20, 50, 0.92)',
214         border: '1px solid rgba(255,255,255,0.2)',
215         borderRadius: 8,
216         color: 'white'
217     }}
218     itemStyle={{ color: 'rgba(255,255,255,0.87)' }}
219 />
220 <Legend
221     formatter={(value) => (
222         <span style={{ color: 'rgba(255,255,255,0.85)' }}>{value}</span>
223     )}
224 />
225 </PieChart>
226 </ResponsiveContainer>
227 )}
228 </CardContent>
229 </Card>
230 );
231 }

```

232

233 export default PieChartView;

src/components/Settings.jsx

```
1 // Settings.jsx
2 // Allows the user to configure a custom URL for fetching exchange rates.
3 // The URL is saved to localStorage and used by all subsequent rate fetches.
4
5 import { useState, useEffect } from 'react';
6 import {
7   Alert, Box, Button, Card, CardContent,
8   TextField, Typography, Divider
9 } from '@mui/material';
10 import { saveRatesUrl, getRatesUrl, fetchAndCacheRates } from '../db';
11
12 /*
13  * Settings renders a simple form where the user can enter and save a
14  * custom exchange rate URL. On save the new rates are fetched immediately
15  * so the rest of the app uses them without requiring a page reload.
16  */
17 function Settings() {
18   // Pre-populate the field with whatever URL is currently configured
19   const [url, setUrl] = useState('');
20   const [message, setMessage] = useState(null);
21
22   // Load the current URL from the db module when the component mounts
23   useEffect(() => {
24     setUrl(getRatesUrl());
25   }, []);
26
27   // Saves the URL and immediately fetches fresh exchange rates
28   const handleSave = async () => {
29     setMessage(null);
30
31     // Trim whitespace before saving
32     const trimmedUrl = url.trim();
33     if (!trimmedUrl) {
34       setMessage({ type: 'error', text: 'Please enter a valid URL.' });
35       return;
36     }
37   }
38 }
```

```

37
38     // Persist the new URL and refresh rates from the server
39     saveRatesUrl(trimmedUrl);
40
41     try {
42         await fetchAndCacheRates();
43         setMessage({ type: 'success', text: 'Settings saved and rates updated successfully!' });
44     } catch (err) {
45         // URL was saved even if the fetch failed ? rates will be retried later
46         setMessage({ type: 'warning', text: 'URL saved, but fetching rates failed. Check the URL.' });
47         console.error(err);
48     }
49 };
50
51 return (
52     <Card elevation={0}>
53         <CardContent sx={{ p: 4, '&:last-child': { pb: 4 } }}>
54             <Typography variant='h5' gutterBottom>
55                 Settings
56             </Typography>
57
58             <Divider sx={{ mb: 3 }} />
59
60             <Typography variant='subtitle1' gutterBottom>
61                 Exchange Rate Source URL
62             </Typography>
63
64             <Typography variant='body2' color='text.secondary' sx={{ mb: 2 }}>
65                 Enter the URL of a JSON endpoint that returns exchange rates in the
66                 following format: `{ "USD":1, "GBP":0.6, "EURO":0.7, "ILS":3.4 }`
67             </Typography>
68
69             {/* Feedback alert shown after a save attempt */}
70             {message && (
71                 <Alert severity={message.type} sx={{ mb: 2 }}>
72                     {message.text}
73                 </Alert>
74             )}
75

```

```

76      {/* URL text field */}
77      <Box sx={{ display: 'flex', gap: 2, alignItems: 'flex-start' }}>
78          <TextField
79              label='Exchange Rate URL'
80              value={url}
81              onChange={e => setUrl(e.target.value)}
82              fullWidth
83              placeholder='https://example.com/rates.json'
84          />
85
86          <Button
87              variant='contained'
88              onClick={handleSave}
89              sx={{ mt: 1, height: 56, whiteSpace: 'nowrap' }}
90          >
91              Save & Refresh
92          </Button>
93      </Box>
94  </CardContent>
95 </Card>
96 );
97 }
98
99 export default Settings;

```